@ER_BRJ

Unit-4 Program Maintenance

Er. Bhisma Raj Joshi
Department of Computer Engineering
[NAST]

OBJECTIVES

@ER_BRJ

- Introduction to Program Maintenance and it’s types
- Problem Areas in Program Maintenance
- Cost issues in software maintenance
- Software errors and its impact
- Program documentation and its standards
- Requirements, Importance and types of documentation
- Program Specification
- System Flow Chart(Elements, Some Examples of Flowcharts)
- Data Flow Diagram, adv and dis-adv of DFD
- Elements/symbols of DFD
- Leveling of DFD
- Examples

@ER_BRJ

EXAM POINT OF VIEW

8HR

2 Long Questions 7+8 =15

1 Short Note 1*5 = 5

Introduction to Program Maintenance

@ER_BRJ

- Program maintenance is the process of modifying, updating, and improving a software system after its initial deployment to ensure its continued functionality, efficiency, and relevance.

- It involves correcting errors, enhancing performance, adapting to environmental changes, and preventing potential future issues.

- Once a software application is developed and deployed, it does not remain static. Over time, users may discover bugs, business requirements may evolve, or new technological advancements may necessitate modifications.

* Types of Program Maintenance

@ER_BRJ

i) Corrective Maintenance

- Corrective maintenance involves identifying and fixing defects or errors in a software system after it has been deployed.

- These defects may include logic errors, incorrect calculations, or performance issues that were not detected during development.

- Example:- Fixing a login issue where users cannot access their accounts. similarly If users encounter a crash when they try to save their work.

- **Process**: Detect -> Diagnose -> Repair -> Test -> Deploy

@ER_BRJ

ii) Adaptive Maintenance

- Adaptive maintenance focuses on modifying software to keep it compatible with changes in its environment. These changes can be due to hardware upgrades, new operating system versions, new laws, or business process updates.

- Example updating a desktop application to work with the latest version of Windows or macOS. similarly modifying an e-commerce website to comply with new tax laws.

- Process: Identify changes in the environment -> Analyze impact -> Plan modifications -> Implement changes -> Test compatibility -> Deploy updated software

@ER_BRJ

iii) Perfective maintenance

- It focuses on improving the software by enhancing its features, performance, or other aspects based on user feedback or evolving requirements. It aims to increase user satisfaction and adapt the software to meet current needs.

Example: Adding new features to the software, optimizing code to run faster, or improving the user interface for better usability.

Process: Gather user feedback -> Analyze needs -> Design enhancements -> Implement changes -> Test improvements -> Deploy updates

@ER_BRJ

iv) Emergency Maintenance

- Emergency maintenance involves urgent, unplanned work required to address unexpected issues or failures in a software system. These issues need immediate attention to restore normal operations and minimize disruption.

Example: Fixing a critical server crash, addressing a sudden security vulnerability, or resolving a major performance issue that affects users.

Process: Detection-->Diagnosis-->Solution Development--> Implementation-->Testing

Problem Areas in Program Maintenance

@ER_BRJ

- Program maintenance is a critical phase in the software development lifecycle, where a program is updated and modified to correct faults, improve performance, adapt to changing environments, or enhance features.
- However, several problem areas can arise during program maintenance, making it a complex and challenging task. Below are the key problem areas in program maintenance:

1. Complexity and Understandability
2. Dependency Management
3. Environmental Changes
4. Change Management
5. Resource and Cost Constraints etc.

@ER_BRJ

1. Complexity and Understandability

- Over time, as more features are added and more bugs are fixed, the code can become increasingly complex and difficult to understand. This is especially true if the original developers are no longer available to explain their code. Similarly poorly documented code makes it difficult for maintainers to understand the program's structure and logic.

2. Dependency Management

- Programs often rely on external libraries and frameworks. If these external components are updated, it can cause integration problems that require significant effort to resolve.

@ER_BRJ

3) Environmental Changes

- When hardware or software gets updated, like a new version of Windows or a faster computer, programs may need changes to keep working properly and run smoothly.

4) Change Management

- Version Control: Managing different versions of the software and ensuring that the correct versions are deployed to the appropriate environments can be complex and costly.

5) Resource and Cost Constraints

- Many organizations face challenges in program maintenance due to limited budgets, tight deadlines, and a shortage of experienced developers. Insufficient funding can restrict necessary updates, while rushed fixes due to time constraints may lead to poor-quality patches. Additionally, the lack of skilled staff makes it even harder to maintain and improve software effectively.

@ER_BRJ

ASSIGNMENT

WHY PROGRAM MAINTENANCE IS NECESSARY? EXPLAIN IN DETAIL ABOUT THE COST ISSUE IN PROGRAM MAINTENANCE.

Impact of Software Errors

@ER_BRJ

- Software errors can cause different problems based on how serious they are, how often they happen, and where they occur. It can cause a significant impact on businesses, users, and overall system performance. Here are some key points:

1. Business and Financial Impact

- Bugs in e-commerce or banking software can lead to transaction failures, resulting in financial losses. and fixing software issues requires time, resources, and skilled personnel, increasing operational expenses.

@ER_BRJ

2. Security and Data Risks

- Bugs or weak security can expose software to hackers, leading to data breaches and system compromises. Security flaws may result in leaks of customer or business data, damaging trust and reputation.

3. User Experience and Reputation Damage

- Frequent crashes, slow performance, or glitches can lead to user dissatisfaction. Poor software quality can harm a company's reputation, leading to loss of customers.

@ER_BRJ

4. Simplified Strategic Impact

- Big errors can push back project deadlines and delay new features, which affects company goals. Important errors take away resources from other projects, causing delays and missed chances for growth.

- Thus software issues can have serious financial, security, operational, and reputational effects. Regular updates, thorough testing, and proactive maintenance are essential to minimize these risks and ensure reliable software performance.

Documentation

@ER_BRJ

- Documentation is the process of collecting and storing the overall activities and operations of the program or software development which performed in every stages of the development. It is a complete record of every aspects of the program or project development.
- This is the process of collecting, organising, storing and maintaining a complete historical record of program and other documents used and prepared during the program or software development stage.
- Good documentation called that type of documentation where every stages of the development with problems and their solution explained clearly and briefly.

Documentation Standard

@ER_BRJ

- Documentation standards are guidelines that ensure software documentation is clear, consistent, and well-organized. They help developers, testers, and users easily understand, use, and maintain the software.

- Good software documentation is essential because poorly documented software can lead to errors, make modifications difficult, and increase maintenance costs. To avoid these issues, many organizations have created documentation standards. These standards provide clear guidelines on how to create well-structured and high-quality documents.

It can be categorized into 3 types which are explained below. @ER_BRJ

1. Process Standard

- Process standards outline the steps for creating documents, including the software tools to use and the quality checks required to ensure accuracy, consistency, and high standards in documentation.

- For example, a company may require all technical documents to be written using Microsoft Word or LaTeX, follow a specific template, and undergo a review process before approval. Additionally, automated tools like Grammarly or Hemingway Editor may be used to check grammar and readability, ensuring clear and professional documentation.

@ER_BRJ

2. Product Standard

- Product standards ensure that all documents created during software development follow a consistent structure, format, and appearance.
- This helps maintain uniformity, making it easier for teams to understand, update, and use the documents efficiently. By following product standards, organizations ensure that documents of the same type have a similar layout, making them professional, readable, and organized.
- For example, Every document should have a predefined structure, title page, introduction, main content, and conclusion. a User Manual always include sections like installation instructions, usage guidelines, troubleshooting tips, and FAQs.

@ER_BRJ

3. Interchange Standards

- Interchange standards focus on the formats used for storing, sharing, and exchanging documents electronically. These standards ensure that documents remain accessible, consistent, and easy to use across different systems and platforms.
- By following interchange standards, organizations can prevent issues like incompatibility, formatting errors, and loss of information when sharing documents between teams, tools, or companies.
- Example: A company may require all official documents to be saved as PDF (Portable Document Format) to preserve formatting and should be accessible regardless of the operating system (Windows, macOS, Linux) or software used.

Requirements of Documentation

@ER_BRJ

- Effective documentation plays a crucial role in ensuring clarity, consistency, and efficiency across various industries, including software development, healthcare, engineering, and business operations. Below are the key requirements of well-structured documentation:

i) Clarity and Conciseness

- Documentation should be easy to understand and straight to the point. Don't use complicated words or technical terms that might confuse people. Keep it simple so everyone can get the message clearly.

@ER_BRJ

ii) Accuracy: Ensure that all information is correct and up-to-date. Inaccuracies can lead to misunderstandings and errors in the processes that rely on the documentation.

iii) Comprehensive Coverage: The documentation should cover all necessary aspects of the subject, leaving no critical information out. It should answer all potential questions that the user might have.

iv) Consistency: Use a consistent format, style, and terminology throughout the documentation. This helps in maintaining a uniform structure and makes it easier for users to follow.

v) Accessibility: Documentation should be easily accessible to those who need it. This includes being available in appropriate formats and ensuring it can be found quickly when needed.

@ER_BRJ

vi) Version Control

- Keep track of changes and updates to the documentation. This helps in managing different versions and ensuring that users have access to the most current information.

- These requirements help ensure that documentation is effective and reliable, serving its purpose in facilitating communication, maintaining standards, and providing essential information.

@ER_BRJ

Types of Documentation

- Documentation can be broadly categorized into several types, each serving a unique purpose. Here are some of the key types:

1) Technical documentation

- It encompasses detailed information about the functioning, design, and usage of a product or system. This type of documentation may be used by developers, testers and also the end users. It consists of documentation of code, algorithm, interfaces and APIs.

@ER_BRJ

2) Project documentation

- It includes all the documents created and used during the lifecycle of a project. This includes project plans, which outline the project's objectives, scope, and timeline; meeting minutes, which record the discussions and decisions made during project meetings; and progress reports, which provide updates on the project's status.

3) Legal documentation

- It includes documents that explain legal responsibilities and agreements. For example contracts, which are formal agreements between parties; licenses, which give permission for specific activities; and compliance documents, which ensure rules and laws are followed.

@ER_BRJ

4. Product documentation

- It includes detailed information about a product. Examples are user guides, which help customers understand and use the product; installation guides, which provide step-by-step instructions for setting up the product; and product specifications, which detail the technical aspects and features of the product.

5. Financial documentation

- This type of documentation includes records related to an organization's financial activities. Like financial statements, which detail income, expenses, and financial position; budgets, which outline projected revenues and expenditures; and audit reports, These documents are crucial for financial transparency and accountability.

@ER_BRJ

ASSIGNMENT

DEFINE PROGRAM SPECIFICATION. WHAT ARE THE KEY COMPONENTS INCLUDED IN A SOFTWARE REQUIREMENTS SPECIFICATION (SRS), AND HOW DO THEY CONTRIBUTE TO THE OVERALL SUCCESS OF A SOFTWARE PROJECT?

System flow Chart

@ER_BRJ

- System flowchart is a type of flowchart that shows the flow of data through a system. It graphically represent how data is input, processed, stored, and outputted.
- These charts are essential for understanding, documenting, and improving systems.

Key Elements of a System Flowchart

Oval: Represents the start or end of a process.

Rectangle: Represents a process or action.

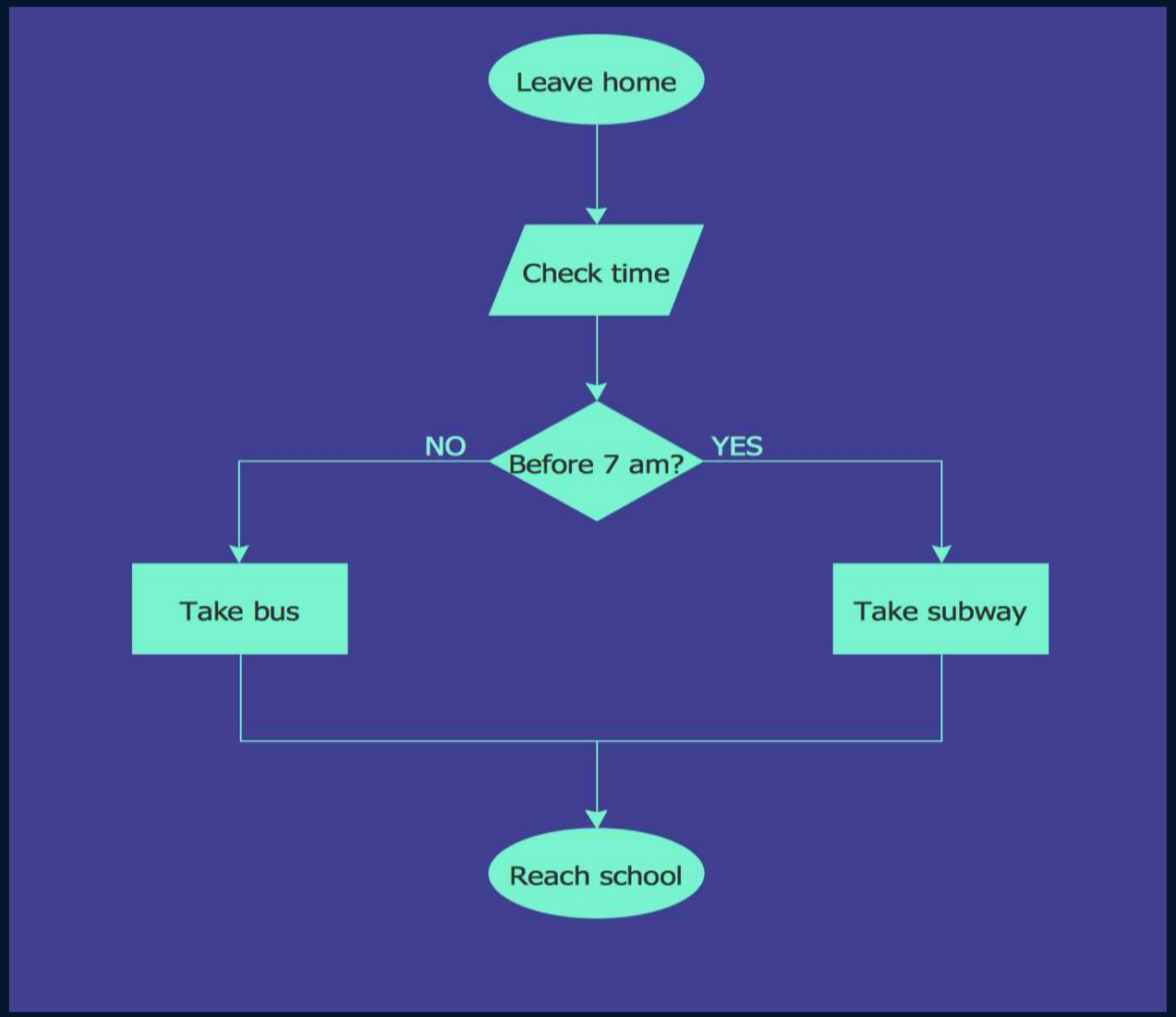
Diamond: Represents a decision point where a choice is made.

Parallelogram: Represents input or output of data.

Arrow: Indicates the direction of flow.

Q.N "Draw a system flowchart for the following scenario: Someone leaves home and checks the time. If it's before 7 am, they take the subway to school. If it's 7 am or later, they take the bus. The goal is to reach school."

@FR_BRJ



@ER_BRJ

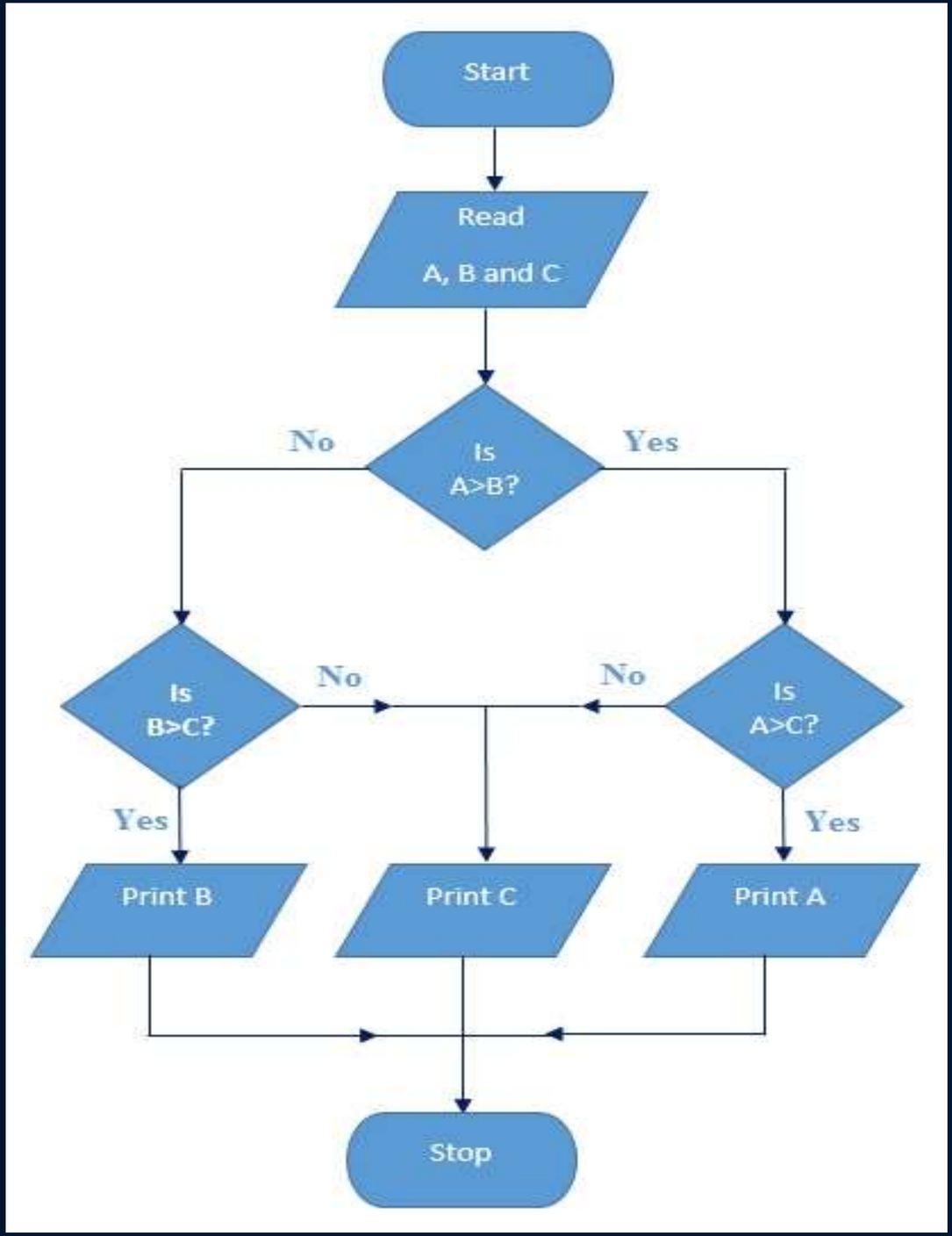


Fig:-System flowchart to find greatest number among 3

@ER_BRJ

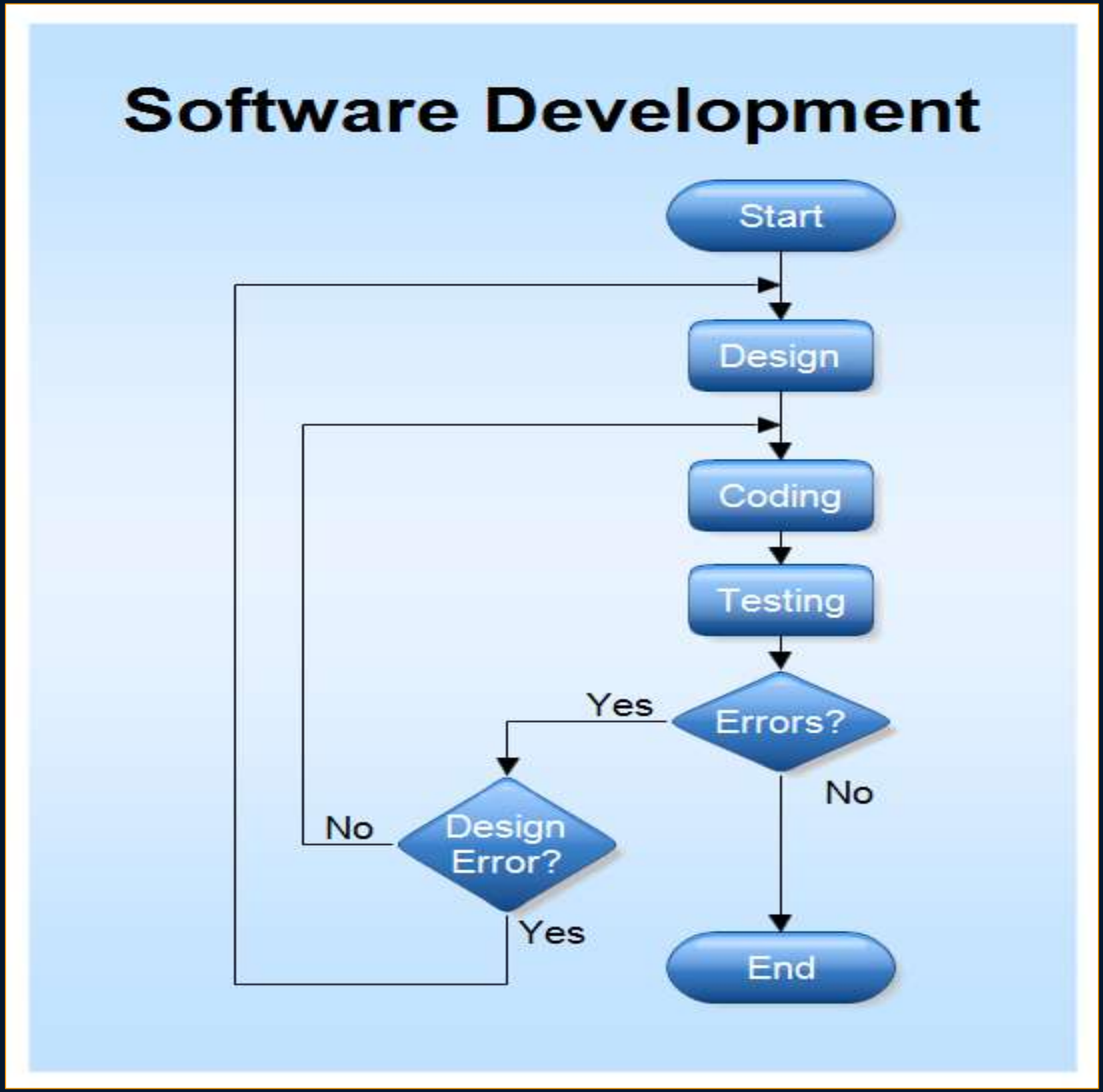


Fig:- system flowchart for software development process

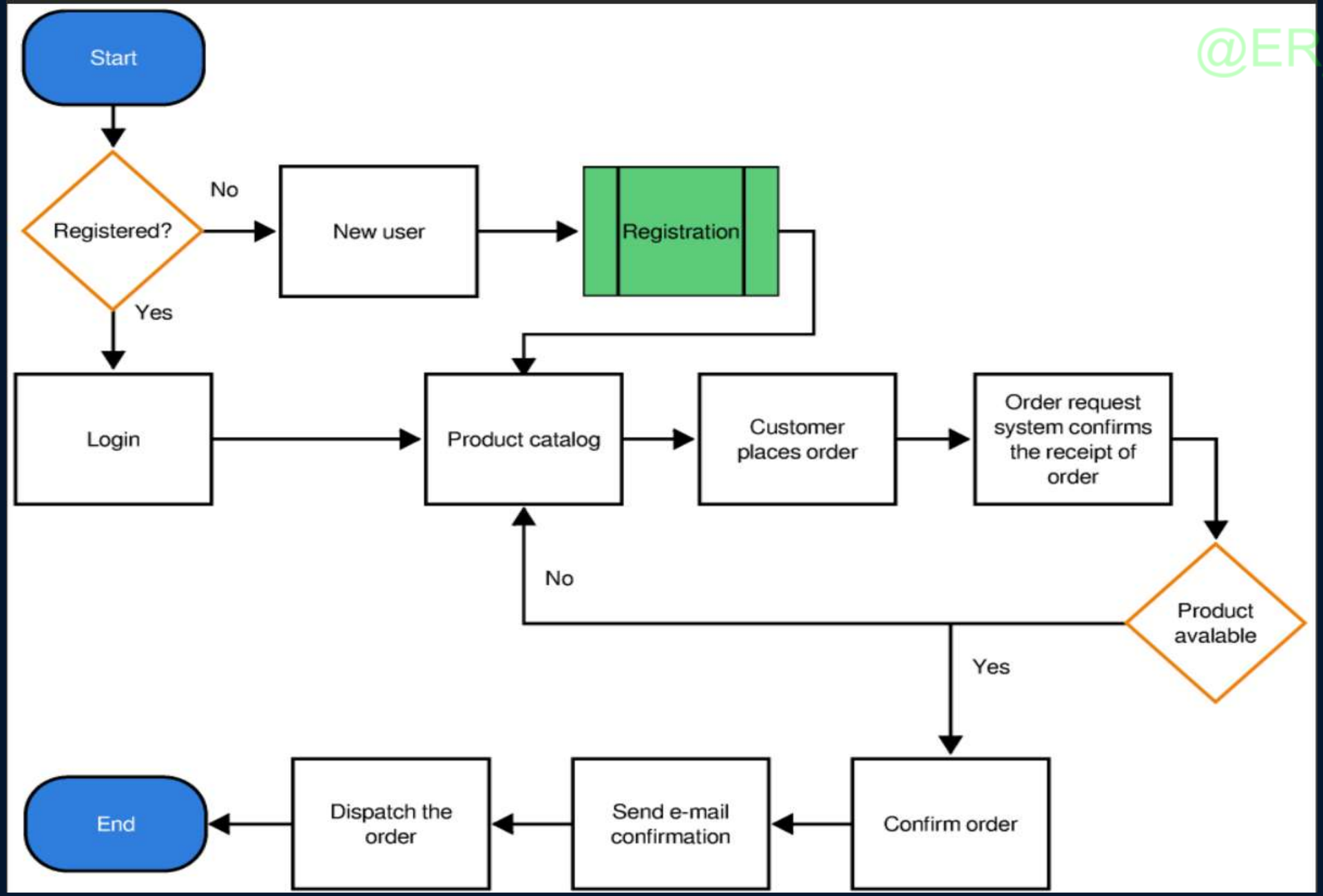


Fig:- System flowchart for Online Shopping

Data Flow Diagram(DFD)

@ER_BRJ

- It is a graphical representation of the flow of data through a system. It is used to visualize how data is processed, stored, and transferred between different components of a system.

- DFDs are commonly used in software engineering, system analysis, and business process modelling to understand, design, or document a system.

- The flowchart shows flow of control in program modules whereas DFDs shows the flow of data in the system at various levels.

@ER_BRJ

- Elements of DFD

- A Data Flow Diagram (DFD) consists of several key elements, each representing a different aspect of the system being modelled.

1. External Entity(Represented by rectangles.)

- These entities are parts of the system that are outside the main process. They send data to or receive data from the system. Since they are not part of the internal workings, they are usually placed on the edges of the diagram.

2. Data Stores

- It represent places where data is held within the system. They are represented as open-ended rectangles (Yourdon and De Marco) or rectangles with a single open-ended side (Gane and Sarson).

@ER_BRJ

3. Processes

- Processes represent the transformation of data within the system. They take inputs (data flows) and transform them into outputs (data flows). Processes are usually represented by circles or ovals (Yourdon and De Marco) or rounded rectangles (Gane and Sarson).

4. Data Flows

- Data flows indicate the movement of data between processes, data stores, and external entities. They are represented by arrows that show the direction in which the data is flowing.

@ER_BRJ

Notation	Yourdon & De Marco	Gene & Sarson
External Entity		
Process		
Data Store		
Data Flow		

Fig:- Symbols used in DFD

@ER_BRJ

Rules for Creating DFDs

- Every process must have at least one input and one output.
- Data flows must be labelled to describe the data being transferred.
- Data cannot flow directly between external entities or data stores; it must go through a process.
- Processes should be named using verbs (e.g., "Calculate Tax").
- Data stores and external entities should be named using nouns (e.g., "Inventory," "Supplier").

Leveling in Data Flow Diagrams (DFD)

@ER_BRJ

- Leveling in DFD is a technique used to break down a complex system into smaller, more manageable parts. It involves creating a hierarchy of diagrams, where each level provides a more detailed view of the system than the previous one.

Purpose of Leveling:

1.Simplify Complexity: Breaking down a system into smaller parts makes it easier to understand and analyse.

2.Focus on Details: Each level focuses on a specific part of the system, allowing for detailed analysis.

3.Improve Communication: Leveling helps in communicating the system's structure and functionality to stakeholders at different levels of detail.

@ER_BRJ

i) Level 0

- It is also known as the Context Diagram: This is the highest level DFD, which provides a broad overview of the entire system.

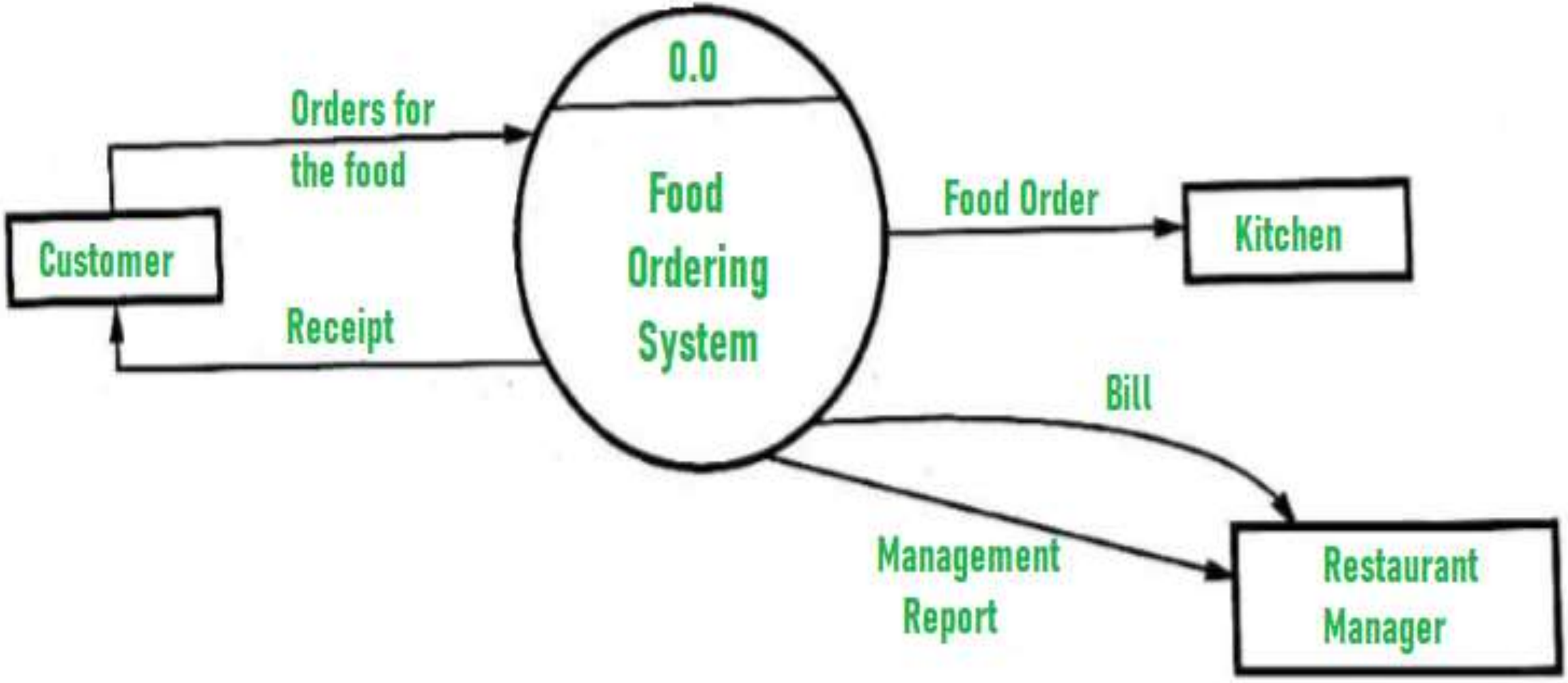
- It includes only one process (representing the entire system), external entities, and data flows between them. The context diagram shows the system's interactions with external entities but no internal details of the system are shown.

- Here are some examples of context diagram

@ER_BRJ

0-LEVEL DFD

@ER_BRJ



Level 0 DFD (Context Level

@ER_BRJ

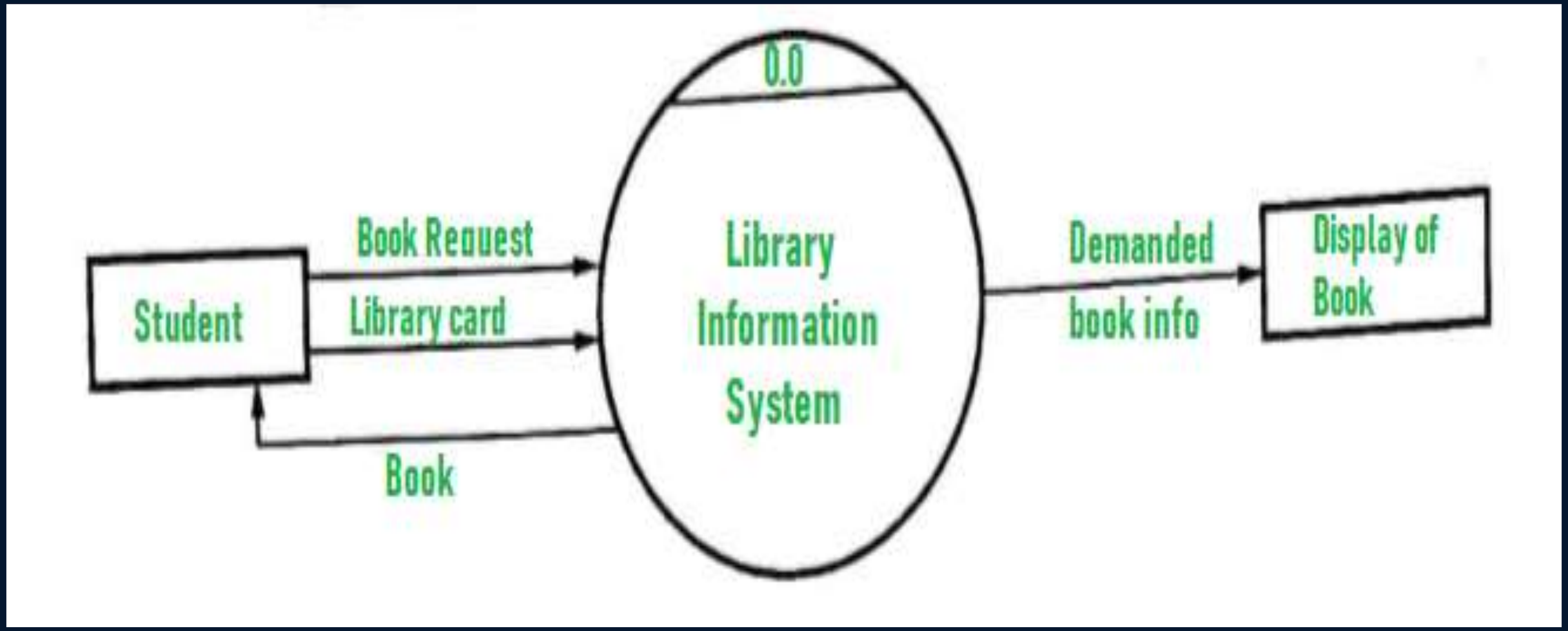


Fig:- Context diagram for Library Information System

@ER_BRJ

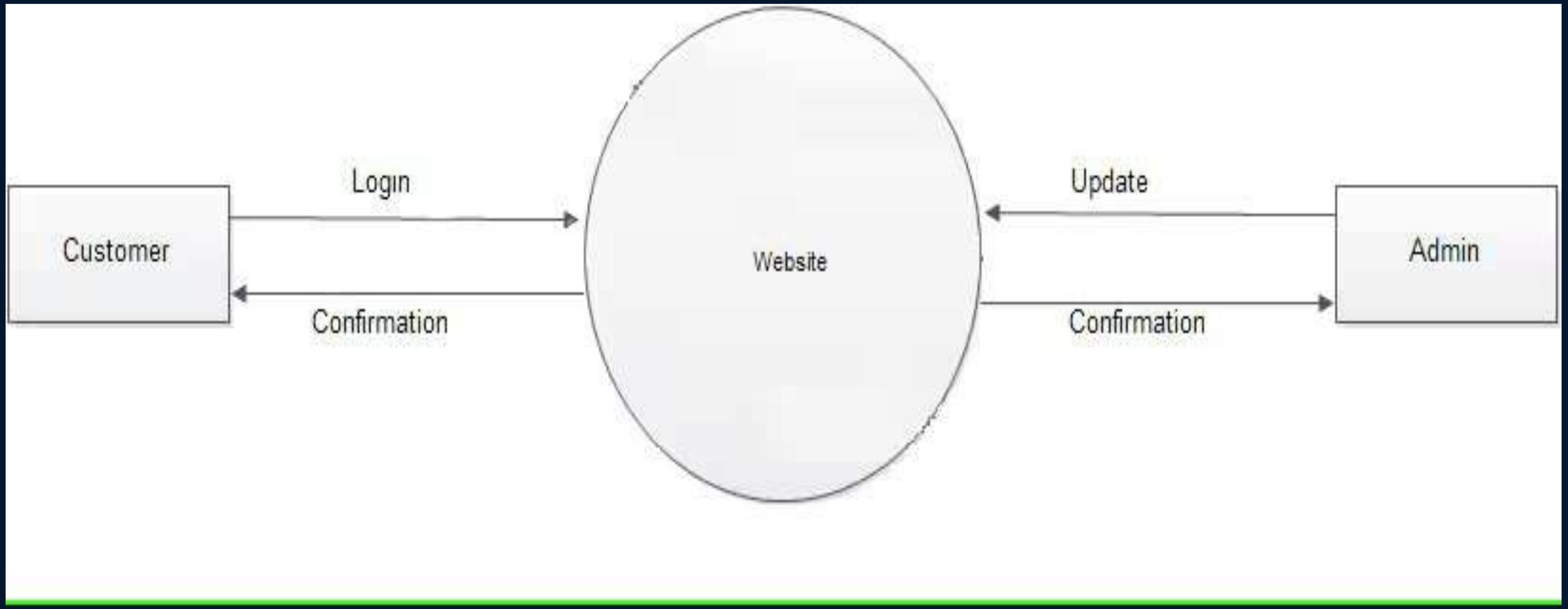


Fig:- level-0 dfd for online shopping System

@ER_BRJ

ii) Level-1 DFD

- This level breaks down the single process from the context diagram into more detailed subprocesses.

- It shows how data flows between these subprocesses, data stores, and external entities. Level 1 DFD provides a more detailed view of the system's internal workings while maintaining readability.

- Here are few examples..

@ER_BRJ

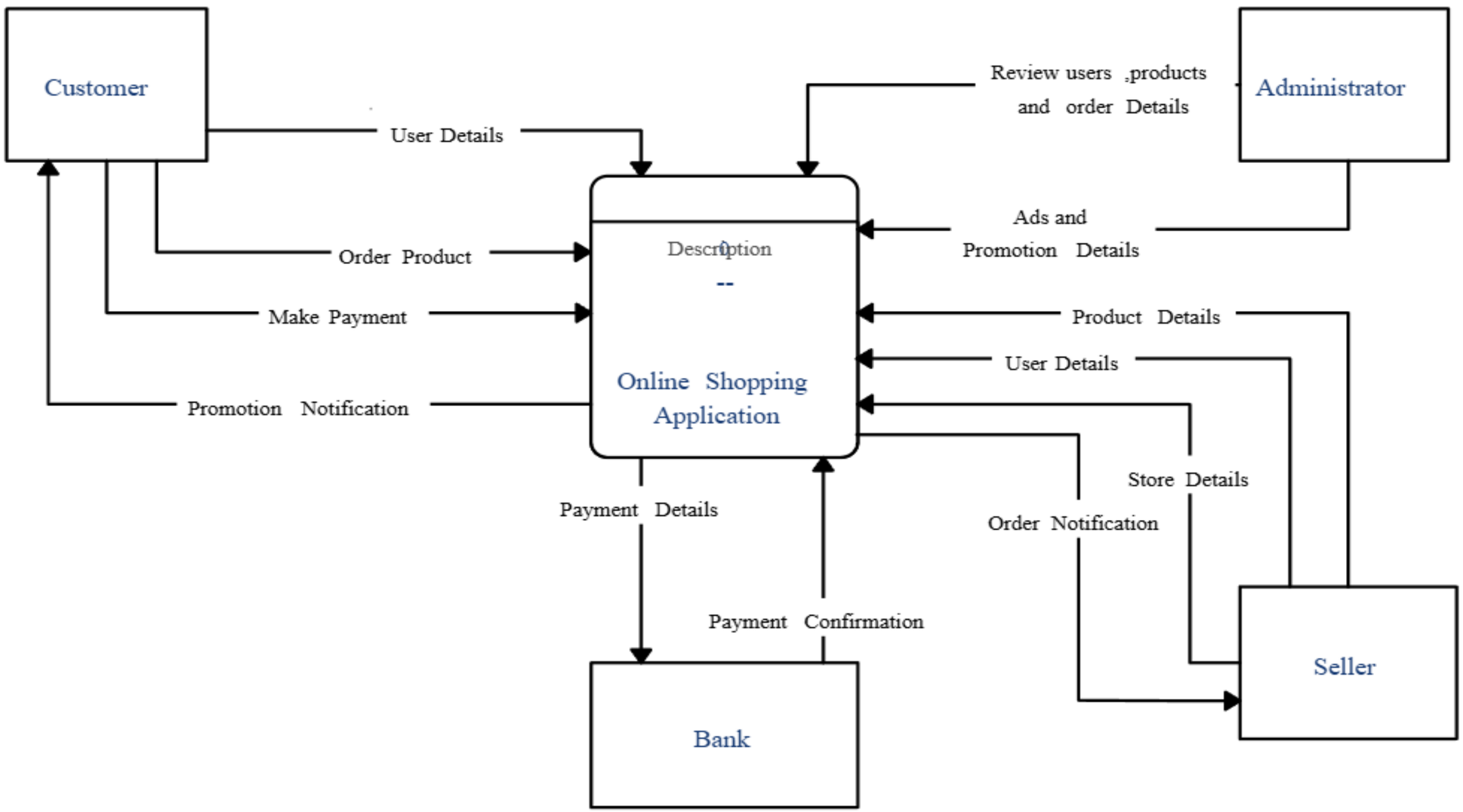


Fig:- Level-1 DFD for online shopping system

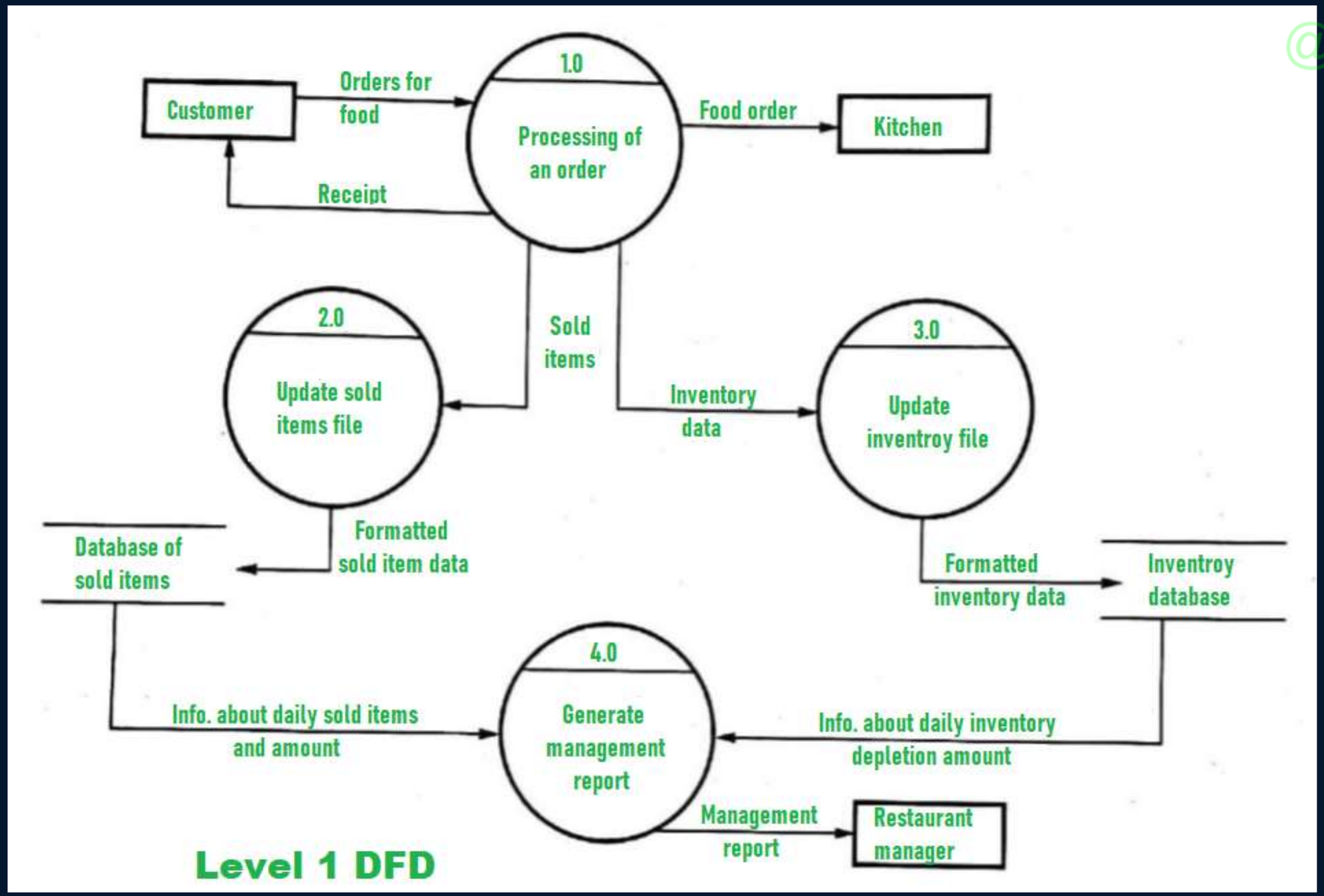


Fig:- level-1 DFD for food ordering system

@ER_BRJ

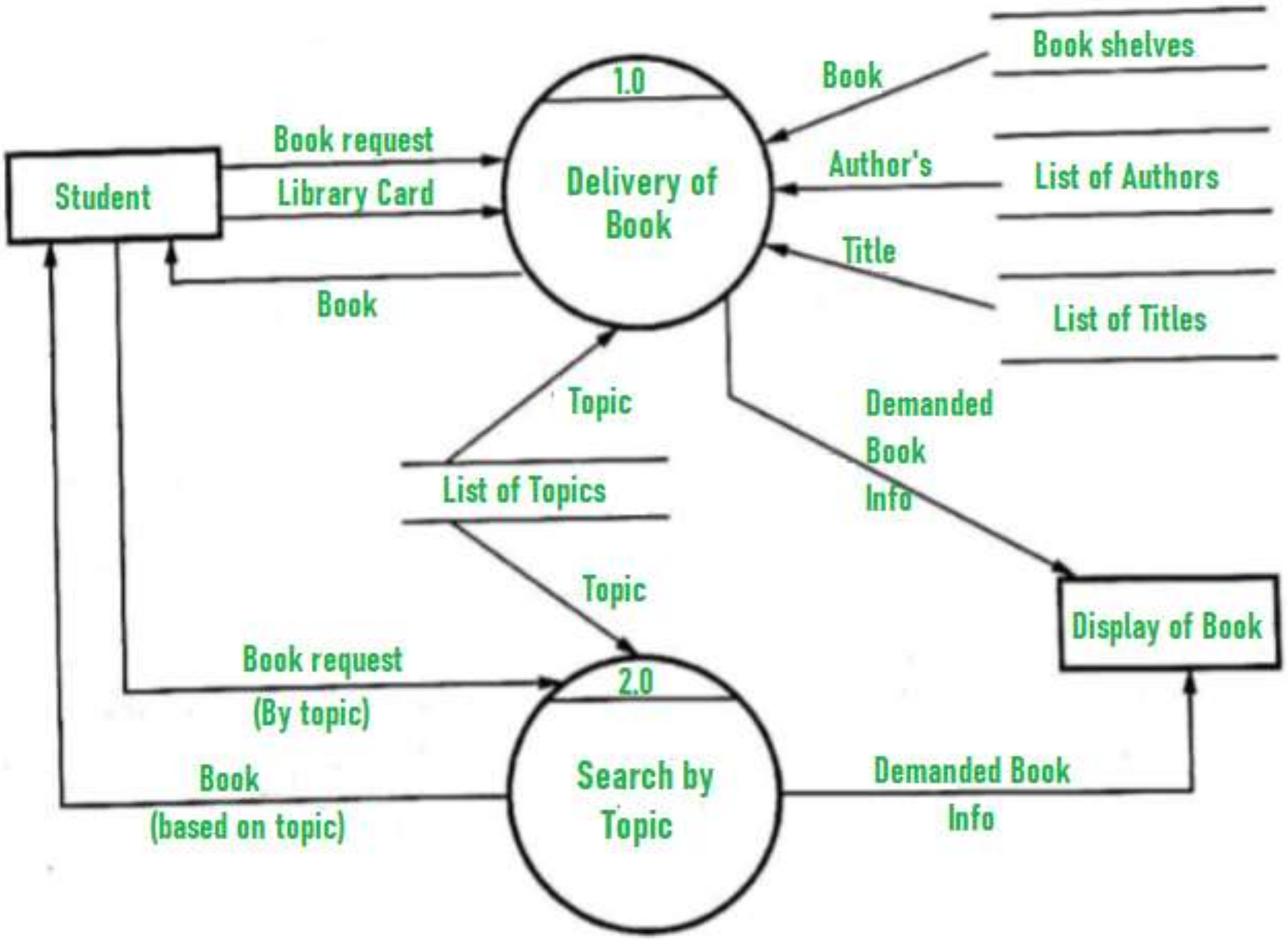


Fig:- Level 1 DFD for library system

@ER_BRJ

iii) Level-2 DFD

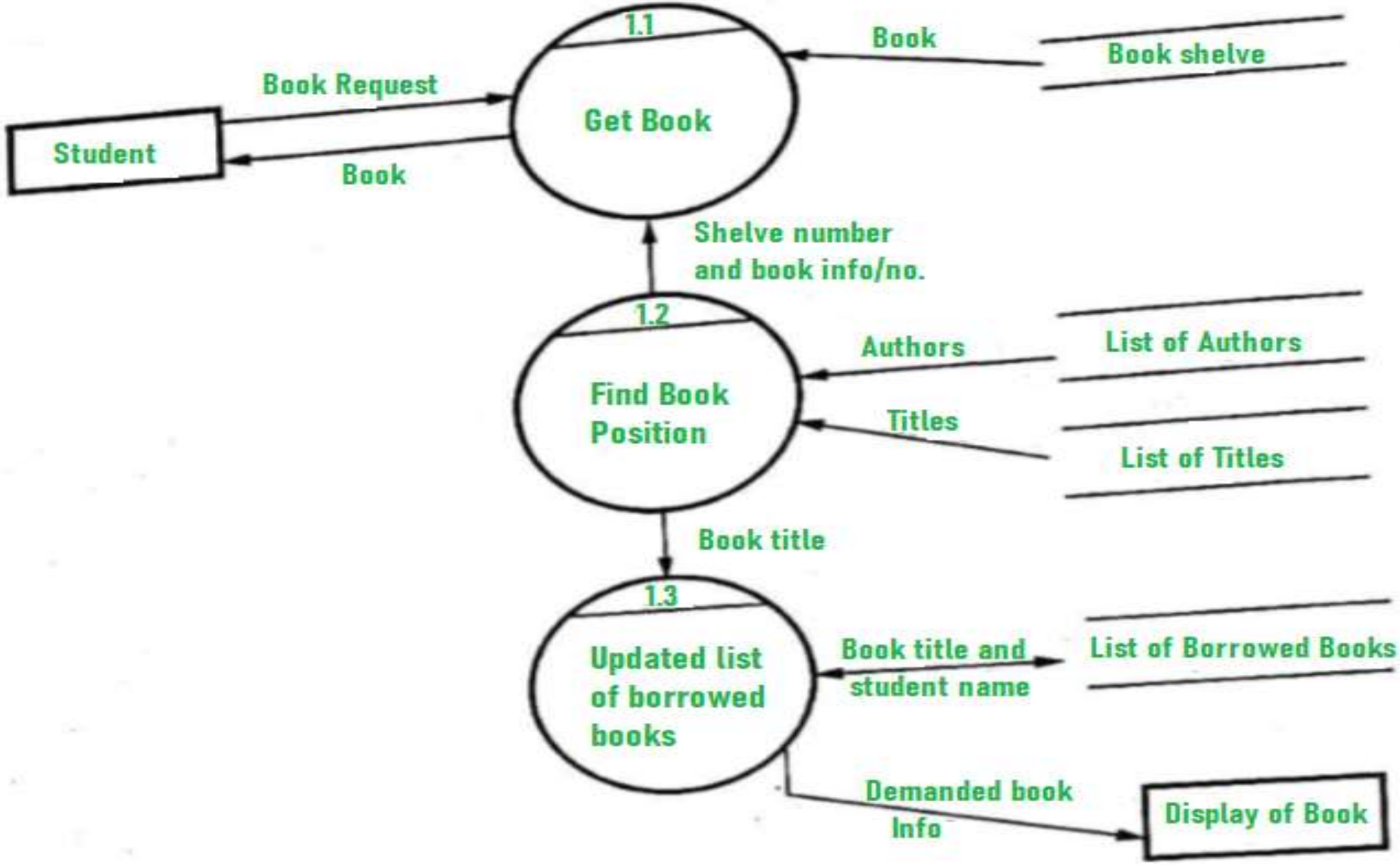
- Lower Levels: As needed, each subprocess from Level 1 can be further decomposed into more detailed subprocesses, creating Level 2 DFDs.

- This decomposition can continue into Level 3, Level 4, and so on, each providing progressively more detailed views of the system.

- Level-2 focuses on specific subprocesses, their data flows, and interactions.

- Here is an example for library management system.

@ER_BRJ



Level 2 DFD

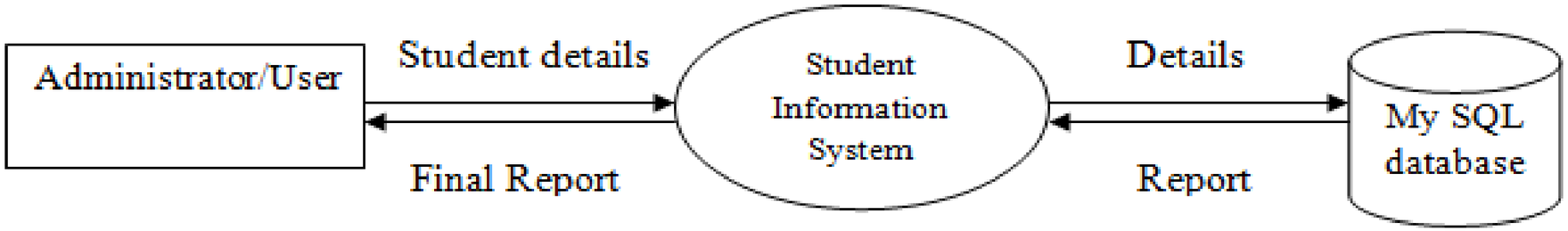
@ER_BRJ

ASSIGNMENT

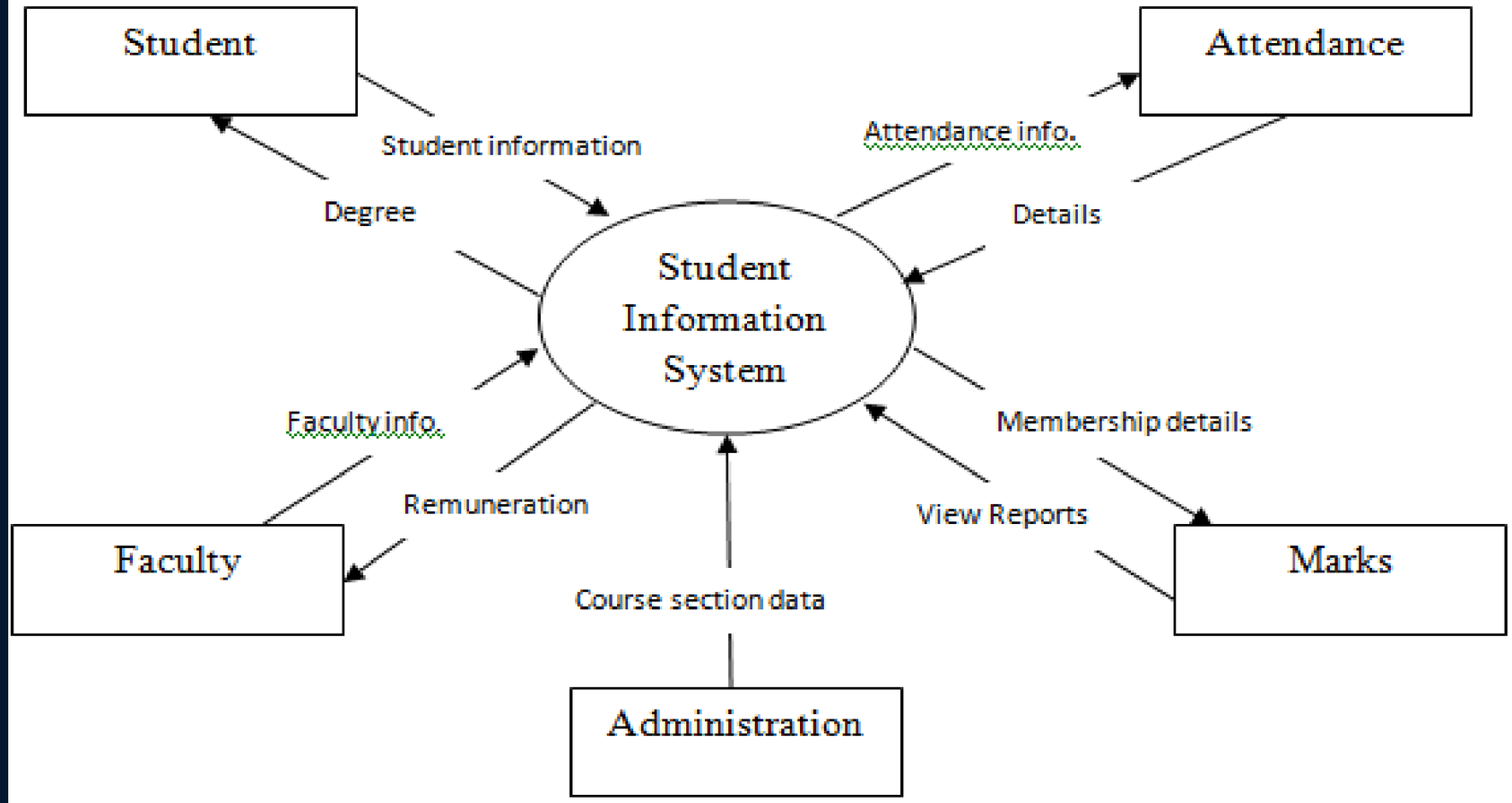
DRAW A CONTEXT DIAGRAM AND LEVEL-1 DFD FOR STUDENT INFORMATION SYSTEM

@ER_BRJ

Level 0 DFD :



Level 1 DFD:



THANK YOU..